

The Work Behind Delegation: A Framework for Supervising AI Coding Agents

YEON SU PARK, School of Computing, KAIST, Republic of Korea

NADIA ARVI, School of Computing, KAIST, Republic of Korea

HAE RI LEE, School of Computing, KAIST, Republic of Korea

SEHOON LIM, School of Computing, KAIST, Republic of Korea

QIANOU MA, Human-Computer-Interaction Institute, Carnegie Mellon University, USA

JUHO KIM, School of Computing, KAIST, Republic of Korea and SkillBench, USA

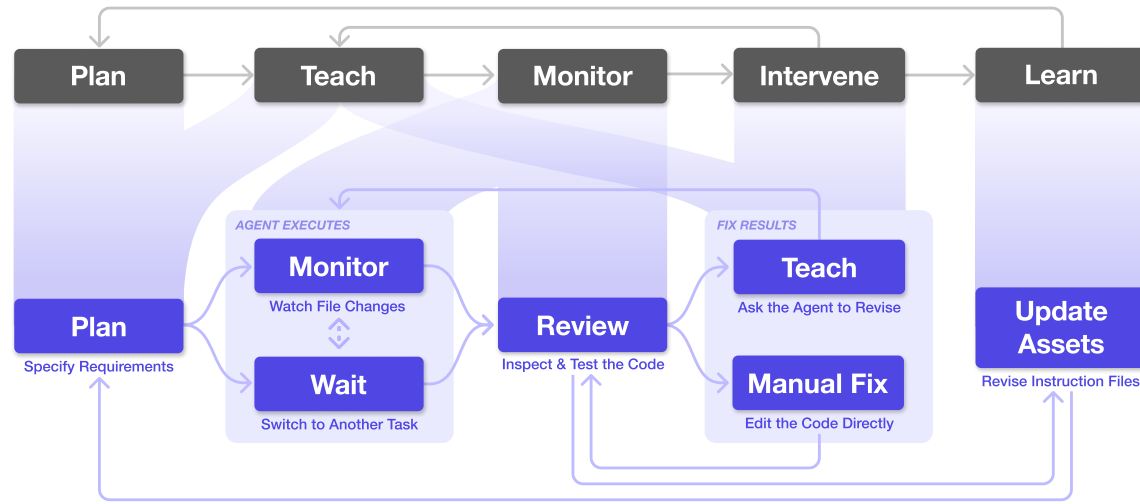


Fig. 1. We introduce a new framework for supervising AI coding agents (bottom) by reconfiguring the five stages of Sheridan's classical human supervisory control (top). Sheridan's five stages—PLAN, TEACH, MONITOR, INTERVENE, and LEARN—map onto our seven stages—PLAN, MONITOR, WAIT, REVIEW, TEACH, MANUAL FIX, and UPDATE ASSETS—as shown by the connecting bands.

As AI coding agents carry out development tasks with greater autonomy, developers are shifting from direct implementation toward supervising delegated work. Yet existing research offers limited understanding of how developers organize supervisory activities into connected workflows. Drawing on observations and workflow diagrams from 19 experienced developers, we reconfigure Sheridan's framework of human supervisory control into seven stages and the connecting loops for supervising AI coding agents. We applied the framework to public developer discussions on Reddit and found that supervisory demands extend across stages and that developers manage them by concentrating effort in planning, delegating supervisory work to other agents, and turning recurring guidance into reusable assets. Our framework provides a useful analytical lens for understanding how developers supervise AI coding agents by capturing how supervision is structured in agentic software development.

CCS Concepts: • **Human-centered computing** → **Empirical studies in HCI**; *HCI theory, concepts and models*.

Authors' Contact Information: Yeon Su Park, School of Computing, KAIST, Daejeon, Republic of Korea, yeonsupark@kaist.ac.kr; Nadia Arvi, School of Computing, KAIST, Daejeon, Republic of Korea, nadia.arvi@kaist.ac.kr; Hae Ri Lee, School of Computing, KAIST, Daejeon, Republic of Korea, harriet@kaist.ac.kr; Sehoon Lim, School of Computing, KAIST, Daejeon, Republic of Korea, sehoon1106@kaist.ac.kr; Qianou Ma, Human-Computer-Interaction Institute, Carnegie Mellon University, Pittsburgh, Pennsylvania, USA, qianoum@cs.cmu.edu; Juho Kim, School of Computing, KAIST, Daejeon, Republic of Korea and SkillBench, Santa Barbara, CA, USA, juhokim@kaist.ac.kr.

Additional Key Words and Phrases: AI Coding Agents, Supervisory Control, Software Development Workflows, AI-Assisted Programming, Human-AI Interaction

1 Introduction

AI coding tools have rapidly expanded their capabilities, moving from assisting developers [62] to carrying out software development tasks with greater autonomy [48, 61]. With the emergence of AI coding agents such as Claude Code [2] and Codex [50], developers can now provide high-level direction and leave the execution to the agent. Given a task description, agents can autonomously plan their own approach, explore the codebase, run commands, and make changes across multiple files [16, 25]. As developers delegate more of this execution to agents, their role shifts from direct implementation toward coordination and oversight [1]. Therefore, developers increasingly act as *supervisors* of AI coding agents, setting the direction of delegated work, tracking progress, and evaluating its output [63].

As developers take on this supervisory role, understanding what effective supervision of AI coding agents requires becomes important. Prior work has examined what developers do at particular points in the process, such as providing clear context and explicit instructions [29], asking agents to explain their changes [7], and providing corrective feedback when execution goes wrong [12]. However, supervision extends across a development task, and effort invested at one point can affect what needs to be monitored, corrected, or reviewed at subsequent points in the task [11, 15, 85]. Given limited human attention and effort, these dependencies highlight the need to consider supervision as a connected process rather than as a set of separate activities. Understanding this process requires an analytical approach that captures how supervisory activities are organized and connected across a task.

Earlier research on automation provides a useful foundation for understanding how human roles change as task execution is increasingly automated by systems such as aircraft autopilots and industrial robots [54, 70]. Among this body of work, we build on Sheridan’s framework of *human supervisory control*, which conceptualizes supervision as a process organized around five stages: PLAN, TEACH, MONITOR, INTERVENE, and LEARN [68, 69]. These stages are linked through recurrent loops, highlighting how one stage shapes subsequent supervisory demands. While Sheridan’s framework was grounded largely in automated systems that executed human-specified goals or instructions, AI coding agents have greater autonomy in determining how to accomplish delegated tasks. Hence, it remains unclear whether Sheridan’s supervisory framework can fully capture supervision under this more agentic form of delegation.

This motivates a central question for our work: *How do developers organize supervision over the course of a task delegated to AI coding agents?* To answer this, we developed a framework for supervising AI coding agents by reconfiguring Sheridan’s framework (Figure 1). The framework was based on a qualitative study with 19 experienced developers, where we observed participants working with agents and asked them to reconstruct their supervision workflows as diagrams. Our analysis identified seven supervisory stages—PLAN, MONITOR, WAIT, REVIEW, TEACH, MANUAL FIX, and UPDATE ASSETS—and loops connecting them. Together, these stages and loops capture how supervision is organized in agentic software development, providing an analytic lens for studying developer supervision of AI coding agents.

We then applied the framework to developer discussions on Reddit to examine supervisory patterns across the development process. We identified three demands that span multiple stages: maintaining alignment between agent behavior and developer intent, enforcing quality beyond task completion, and maintaining understanding of agents’ work. We also found three strategies for managing these demands: concentrating effort in planning, distributing supervisory work to other agents, and externalizing repeated supervision into reusable assets. Through the framework, we show how supervisory demands extend across stages and how developers redistribute their effort in response.

The contributions of this paper are as follows:

- A framework for supervising AI coding agents that consists of seven stages and the loops connecting them.
- Findings from applying the framework to examine supervisory demands and the strategies developers use to manage them across the development process.

2 Related Work

We review prior work to contextualize our framework for developer supervision of AI coding agents: (1) AI-assisted software development, (2) frameworks for human-agent collaboration, and (3) human roles in the agentic era.

2.1 AI-Assisted Software Development

AI assistance in software development has shifted from providing localized code suggestions [18, 45, 57, 64] toward increasingly autonomous AI agents that can carry out development tasks with limited human intervention [19, 40, 60]. As AI systems have become more capable [16, 21], how developers interact with them has also changed.

Prior work has examined AI assistance in software development and its integration into developer workflows. Early studies focused on developers’ interactions with individual AI-generated suggestions, examining how they evaluated, accepted, and modified suggestions during programming [5, 48]. More recent work has expanded attention to AI use across programming sessions and broader development workflows, drawing on interviews with practitioners [15, 76], interaction logs from real-world use [75, 80], and longitudinal telemetry [67]. Together, these studies highlight the importance of understanding developer-AI interaction as AI becomes deeply integrated into software development.

Agentic AI introduces new dynamics of developer-AI interaction. AI agents can proactively plan and execute multi-step actions [12], incorporate broader contextual information [25], and produce artifacts across development workflows [84]. As developers delegate more execution to agents, supervision becomes increasingly important, yet its organization across a development task remains underexplored. Therefore, in this work, we examine the supervisory activities developers perform and how these activities connect throughout the process.

2.2 Frameworks for Human-Agent Collaboration

A growing body of empirical work has examined human-agent collaboration through interviews [15], observational studies [29, 71], and real-world interaction logs [7, 75]. Building on these findings, researchers have developed different conceptualizations of human-agent interaction, capturing recurring modes, activities, and dimensions of collaboration. For example, Barke et al. [5] distinguish between exploration and acceleration modes of interaction with programming assistants, while Dhanorkar et al. [15] identify four forms of oversight: a priori control, co-planning, real-time monitoring, and post-hoc review. Other studies describe human-agent interaction in terms of the degree of human involvement [12], time allocation across activity states [48], and the distribution of initiative between human and system [46].

Beyond these study-specific conceptualizations, recent work has proposed more explicit frameworks for human-agent collaboration. Feng et al. [21] structure collaboration around levels of agent autonomy and corresponding user roles, while Wang and Lu [79] propose a process-oriented framework that connects interaction, process, and infrastructure. These frameworks provide systematic ways to structure human-agent collaboration, but offer limited guidance to collaboration centered on supervising delegated agent work. In this work, we develop a framework that captures the supervisory activities involved and how they connect across a development task.

2.3 Human Roles in the Agentic Era

The emergence of AI as a collaborator is not unique to software development. Similar shifts toward human–AI collaboration have been observed across domains such as design [10, 81], writing [39], and knowledge work [83], where AI systems have become integrated into everyday workflows. As these systems become more agentic and take on greater responsibility for execution, human roles are being reconceptualized. Rather than acting as primary executors while AI provides passive assistance, humans are increasingly shifting toward supervisory roles over delegated work by defining goals [22, 44, 76], evaluating outcomes [20], and intervening when necessary [85].

These shifts also align with longstanding perspectives in automation research, which describe how responsibilities shift between humans and systems as system autonomy increases [54, 70]. Taken together, these perspectives point toward a shift in human contribution from direct execution toward higher-level activities such as providing context, exercising judgment, and guiding AI-generated work [13, 20]. Building on these observations, we examine how this shift manifests in software development as developers increasingly supervise work delegated to AI agents.

3 Background

In this section, we provide background on Sheridan’s *human supervisory control* framework, which serves as the theoretical starting point for our work examining how developers supervise AI coding agents. The *human supervisory control* framework describes how a human supervisor directs and oversees automated systems such as aircraft autopilots and industrial robots [68–70]. The framework consists of five supervisory stages: PLAN, TEACH, MONITOR, INTERVENE, and LEARN, connected by recurring loops as illustrated in Figure 2. We briefly explain each stage below.

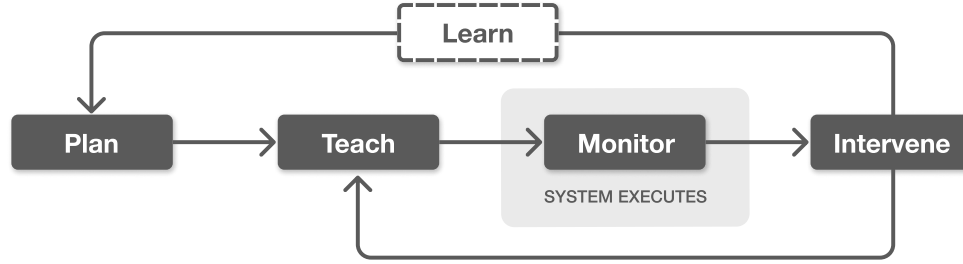


Fig. 2. Sheridan’s framework of human supervisory control, consisting of five stages—PLAN, TEACH, MONITOR, INTERVENE, and LEARN—connected in sequence, with an inner loop from INTERVENE to TEACH and an outer loop from INTERVENE through LEARN to PLAN.

The supervisor begins with PLAN, where they develop an understanding of the process and system capabilities, then determine the goals, constraints, and overall strategy. In TEACH, these plans are translated into instructions for the automated system. During execution, MONITOR involves observing the system and assessing whether it is progressing as intended, while INTERVENE occurs when the supervisor needs to revise the instructions or take more direct control. Revising instructions returns the process to TEACH, whereas satisfactory completion leads to LEARN, where the supervisor reflects on key events and prior decisions to inform future PLAN activities. Together, these stages form recurring control loops in which supervision can return to earlier stages as execution progresses (Figure 2).

4 Method

We conducted a qualitative study with experienced software developers to develop a framework for supervising AI coding agents. We used Sheridan’s framework of human supervisory control [68, 69] (§3) as a theoretical starting point to examine how its stages and loops translate to the supervision of AI coding agents as developers delegate more implementation to them. We observed participants working on their own coding tasks with their usual agent setups and asked them to reconstruct their supervision workflows as diagrams. We analyzed these workflow diagrams together with task observations and think-aloud data to develop our framework, which we introduce in §5.

In this section, we describe how we designed and conducted the study in detail. The study was reviewed and approved by the Institutional Review Board (IRB) of our institution prior to conducting the study.

4.1 Participants

We distributed a screening survey through university forums and developer communities in South Korea, as well as LinkedIn groups and networks for international recruitment. The survey collected information about respondents’ software development background, AI coding agent use (i.e., which agents they used, how frequently they used them, and the types of tasks they delegated), and the coding task they could bring to the study session.

Based on the survey responses, we applied three eligibility criteria: (1) at least three years of professional software development or graduate-level research experience in a software-intensive field (e.g., computer science, electrical engineering), (2) regular use of AI coding agents for substantive development tasks in professional, research, or other real-world projects, and (3) the ability to bring a shareable coding task with a clearly defined goal for a 40-minute session. These criteria ensured that participants had sufficient development experience to judge agent output critically and sufficient familiarity with agents to have established supervisory practices of their own.

We recruited 20 participants who met these criteria. One participant was unable to use their regular agent setup during the session and was excluded from the analysis, resulting in a final sample of 19 participants. The full list of participants and their information is shown in Table 1. We deliberately recruited participants across diverse roles and development backgrounds to capture variation in how developers supervise AI coding agents.

Sessions were conducted in Korean or English, and Korean transcripts were translated into English for analysis. All eligible participants received either a 120 USD Amazon gift card or 150,000 KRW ($\approx$ 98 USD) for a 90-minute study.

4.2 Task

Following Huang et al. [29], participants were asked to bring their own task from their ongoing project rather than complete a given standardized task. This allowed us to observe realistic development workflows [8], while introducing natural variation in tasks, codebases, and agent configurations across the study sessions [36].

Each task needed a clearly defined goal that could be pursued within approximately 40 minutes. This scope allowed us to observe supervision end-to-end within a single session. Tasks could come from ongoing work, open-source projects, side projects, or research code, as long as they could be shared with the research team and did not contain confidential or personal information. The *Task* column in Table 1 summarizes the task brought by each participant.

Table 1. Overview of the 19 participants and their study sessions. In the Agent Setup column, “+” denotes an integrated setup, “&” denotes two agents used independently, and “in” denotes an agent running within an orchestration environment.

ID	Current Role	Experience (Years)	Agent Setup	Task
P1	Frontend Developer	26	GitHub Copilot	Adding recommendation comparison pages
P2	Graduate Student	4	Claude Code	Generating publication-ready research figures
P3	Backend Developer	5	Codex + Lazy Codex	Building a web game prototype
P4	Product Manager	6	Claude Code + Superpowers	Pivoting a scheduler into a contact manager
P5	Graduate Student	3	Codex + OhMyCodex	Refining a 3D simulation app UI and docs
P6	Backend Developer	6	Codex	Fixing trading backend order bugs
P7	Full Stack Developer	5	Claude Code in Cmux	Adding tests to a web service
P8	Graduate Student	4	Codex	Adding NPC movement and dialogue
P9	Full Stack Developer	3	OpenCode + OhMyOpenCode	Adding diagram support to a math generator
P10	Indie Game Developer	5	Codex + Unity MCP	Adding currency UI and sound effects
P11	Backend Developer	7	Codex	Building a chess mistake-analysis website
P12	ML Engineer	7	Claude Code in Conductor	Generating labels for video augmentation pairs
P13	Data Analyst	5	Codex	Adding mood-based recommendations
P14	Frontend Developer	5	Codex + Claude Code	Adding an Android reflection widget
P15	Graduate Student	5	Hermes & Claude Code	Optimizing load time and accessibility
P16	Full Stack Developer	3	Claude Code	Building a bid-document analysis website
P17	Full Stack Developer	6	Claude Code	Migrating a blog backend to PostgreSQL
P18	Full Stack Developer	6	Claude Code	Adding dashboard and delete features
P19	Founding Engineer	15	Antigravity	Migrating a remote-control bridge and CLI

4.3 Procedure

We conducted the study remotely over Zoom¹. Each session lasted approximately 90 minutes and consisted of three phases: (1) a pre-task interview, (2) a think-aloud coding task, and (3) a supervision workflow modeling activity. All sessions were recorded and transcribed for later analysis. We describe each phase below.

4.3.1 Pre-Task Interview. We began with a semi-structured interview that expanded on participants’ initial survey responses about their development background and agent use. Participants then introduced their prepared task and the concrete goal they intended to pursue during the session. We provided brief instructions on the think-aloud protocol as needed. This phase lasted approximately 10 minutes. The full interview questions are provided in Appendix A.

4.3.2 Coding Session. In this phase, participants worked on their prepared task in their own environment, sharing their screen(s) throughout. They were asked to think aloud as they worked, and we prompted them when they remained silent for an extended period. Sessions ended when participants felt they had reached their goal, or after approximately 40 minutes had passed. In the latter case, we briefly asked how much work remained.

4.3.3 Workflow Design Session. Participants then drew a diagram of their supervision workflow on a shared online whiteboard. To examine how human supervisory control translates to AI coding agents, we first introduced Sheridan’s

¹<https://zoom.us/>

framework to participants. We emphasized that participants did not need to follow Sheridan’s stages and asked them to construct their diagrams based on their own supervision practices. The activity proceeded in three steps.

First, participants reflected on the task they had just completed and divided their supervision into stages they considered meaningful and distinct. If a stage was difficult to name concisely, they could write a short description instead. Second, they arranged the stages in order and marked recurring transitions as loops. Finally, they considered how the diagram might extend to their other tasks and to other developers, revising it accordingly. As in the coding session, participants were asked to think aloud during the activity, explaining the meaning of each stage and loop and the reasoning behind it. This phase lasted approximately 30 minutes. Example diagrams are provided in Appendix B.

4.3.4 Inductive Content Analysis. We analyzed participants’ workflow diagrams, task observations, and think-aloud data following the content analysis approach of Elo and Kyngäs [17]. We inductively derived stages and loops in participants’ supervision processes and used the findings to reconfigure Sheridan’s framework for AI coding agents.

After the first eight sessions, the three authors agreed that common patterns were emerging across participants. Each author independently reviewed all eight participants’ data to familiarize themselves with the data. We then collaboratively coded each participant’s supervision process across the three sources. The diagrams provided participants’ own representations of their processes, while task observations and think-aloud data clarified ambiguities and captured omitted steps. Through iterative comparison, we grouped codes representing similar supervisory functions, aligned their terminology and granularity, and organized them into an initial framework of stages and loops.

We first checked the framework against the first eight cases. After each of the remaining 11 sessions, we compared the participant’s workflow with the framework and examined whether stages, transitions, or loops required revision. When a mismatch arose, the three authors revisited the data, discussed the case, and revised the framework as needed. This process led to the removal of one stage and the addition of one loop. As no new stages or loops emerged in later sessions, we stopped data collection after 19 participants. The final framework consists of seven stages—PLAN, MONITOR, WAIT, REVIEW, TEACH, MANUAL FIX, and UPDATE ASSETS—and the loops connecting them. We present it in detail in §5.

5 A Framework for Supervising AI Coding Agents

We developed a new framework for supervising AI coding agents that reconfigures Sheridan’s framework of human supervisory control based on our empirical findings. Our framework identifies seven distinct supervisory stages: PLAN, MONITOR, WAIT, REVIEW, TEACH, MANUAL FIX, and UPDATE ASSETS. These stages are connected through transitions and loops that capture the overall process of developer supervision across a development task, as illustrated in Figure 3.

In this section, we introduce our framework by describing the stages in the general order they appeared in participants’ workflows, grouping together those with similar supervisory goals. For each stage, we explain what developers did, how it relates to Sheridan’s framework, and how it connects with other stages.

PLAN. PLAN corresponds closely to Sheridan’s planning stage, in which the supervisor sets a goal and an overall approach before execution begins. At this stage, developers define the work to be delegated to the agent and establish its boundaries in advance, specifying goals, requirements, constraints, success criteria, and an intended approach. For example, P10 described, “I usually write three things [in the plan]: the situation, the success criteria, and the cautions. Here is the situation I am in, here is what you need to implement, and here is what you need to watch out for.”

In Sheridan’s framework, the supervisor determines the task and approach during PLAN, then separately translates it into commands the system can execute during TEACH. With AI coding agents, this translation was often unnecessary.

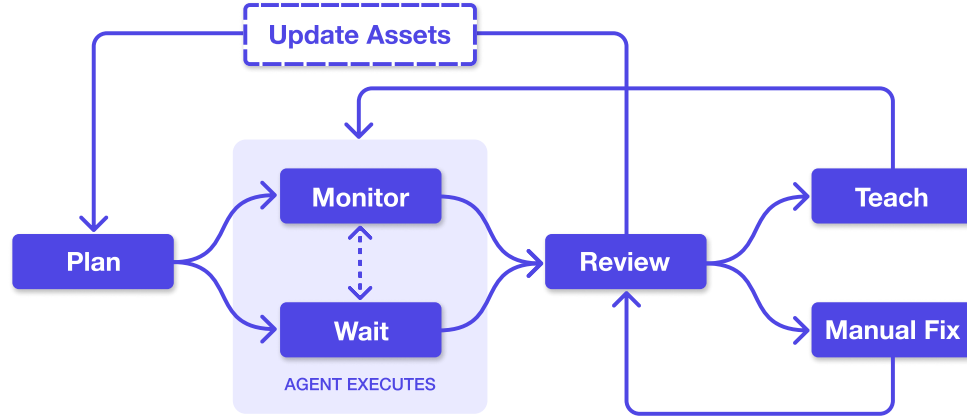


Fig. 3. Our framework for supervising AI coding agents, consisting of seven stages—PLAN, MONITOR, WAIT, REVIEW, TEACH, MANUAL FIX, and UPDATE ASSETS—and the loops connecting them. MONITOR and WAIT occur while the agent executes, and developers can alternate between them. From REVIEW, developers can go to either TEACH or MANUAL FIX. TEACH loops back to MONITOR/WAIT, while MANUAL FIX loops back to REVIEW. UPDATE ASSETS is an optional stage that feeds learnings from REVIEW back into the next PLAN.

Developers write the plan down in natural language with sufficient detail for the agent to act on directly. P3 explained, “*I did planning, and the planning itself was the teaching. I wanted the teaching step to happen inside it.*” Hence, PLAN serves as both the developer’s specification and the agent’s initial instruction, absorbing the initial TEACH step.

In our framework, PLAN also forms a self-loop. Developers refine their goals, requirements, and approach over multiple iterations before execution begins. These iterations include the developer’s own revisions as well as input from the agent or another AI, which proposes, elaborates, or critiques the plan. P18 said, “*There’s a loop here, so if I don’t like it I keep going around, revising [with AI] until I’m satisfied. And if the plan is right, then I assign the work.*” The loop ends when the plan aligns with the developer’s intent and is detailed enough for the agent to act on directly. Developers then hand it over, and the workflow moves to MONITOR or WAIT as the agent begins executing.

MONITOR & WAIT. MONITOR corresponds closely to Sheridan’s monitoring stage, where supervisors observe an automated system during execution to track its progress and detect potential problems. Developers similarly observe agents as they work by reading their reasoning, watching the commands and file changes they produce, and checking intermediate outputs as they appear. P1 explained, “*Definitely did the monitoring of the process to check if there are any easily identifiable errors happening in the reasoning or in the overall process.*” In our framework, MONITOR forms a self-loop similar to Sheridan’s monitoring loop. Developers continue observing the same ongoing execution and sometimes send additional prompts in response to what they see, without stopping the agent.

However, developers do not always monitor agents continuously during execution. When an agent can continue working for extended periods without human input, participants sometimes left it running and turned their attention elsewhere, describing this as a distinct WAIT stage. They explicitly distinguished these periods from MONITOR. During WAIT, some developers shift to supervising agents on separate tasks, while others turn to unrelated activities. P2 explained, “*I instruct from the planning stage, then wait, and while waiting I go on to another round of planning and instructing.*” On the other hand, P9 said, “*I do something else—other work, or I just watch YouTube.*”

MONITOR and WAIT can alternate within the same execution as developers shift their attention between the agent and other activities. P3 described, “*Sometimes I read them and sometimes I don’t. I read for a bit, then go do something*

else, and then when I read and something seems really off, I stop it.” When execution finishes, developers move to REVIEW. When monitoring reveals a fundamental problem, they stop the execution and begin a new cycle from PLAN.

REVIEW. REVIEW is a new supervisory stage in which developers assess the agent’s completed work against their intent and requirements. While Sheridan includes such assessment within MONITOR, participants treated it as a separate REVIEW stage. Developers inspect the code and diffs, run tests, or interact with the implemented feature to assess whether the changes behave as intended and satisfy their requirements. P18 explained, “[The plan] cannot capture all of my intent—some things get compressed, or it fills them in by guessing, and those only become visible once you get to this [Review] stage. [...] What I look for in [Review] is the gap between my intent and the result.”

REVIEW also forms a self-loop because developers rarely evaluate the agent’s work in a single pass. Instead, they continue checking it in multiple ways until they are confident that it meets their expectations. For example, P9 described, “I first check how many of the tests pass—the acceptance conditions I gave the AI and the tests that match them. When those pass, I then check the feature with my own eyes, monkey-testing it.” When developers are satisfied with the result, they complete the current task and move to PLAN for a new task, sometimes passing through *Update Assets* first. When they find errors or aspects they are dissatisfied with, they move to TEACH or MANUAL FIX.

TEACH & MANUAL FIX. When REVIEW reveals errors or mismatches with the intent, developers supervise the correction in two ways. They either redirect the agent through TEACH or take over the correction themselves through MANUAL FIX. In Sheridan’s framework, TEACH translates an intended action into commands that the automated system can execute, following either initial planning or corrective intervention. With coding agents, the initial teaching function is absorbed into PLAN, leaving only corrective instructions as a distinct TEACH stage (see Figure 1).

TEACH takes the form of follow-up prompting. Developers prompt the agent again based on problems identified during REVIEW, adding missing details, correcting misunderstandings, reporting errors, or specifying how the work should change. P14 described this process, “Most of the time, it’s because I didn’t communicate something. [...] I realize, ‘I left this out,’ and tell [the agent] again. [...] If there’s actually a bug, I just tell it there’s a bug, and it usually fixes it.”

MANUAL FIX is where developers choose to take over and do the correction themselves. Developers make the correction when the change is simple or faster to carry out directly, or when they expect further prompting to be inefficient. P6 explained, “I could have had [the agent] do it, but it was faster for me, so I just changed it by myself.”

After TEACH, developers return to MONITOR as the agent resumes execution, and then to REVIEW, while MANUAL FIX directly leads back to REVIEW. In both cases, the loop repeats until developers are satisfied with the result in REVIEW.

UPDATE ASSETS. UPDATE ASSETS is an optional stage that extends supervision beyond the current task by updating persistent resources that can guide future interactions with the agent. These resources include files such as AGENT.md, project documentation, and other shared assets that record project knowledge, preferences, constraints, or lessons from prior work. Developers do not necessarily update these assets after every task, but when they do, the updated assets support future tasks, beginning with a new PLAN stage. In this way, Sheridan’s LEARN, which focuses on updating the supervisor’s understanding from experience, becomes UPDATE ASSETS as developers externalize what they have learned into persistent resources that can guide the agent in future work. P4 explained, “[Learning] also happens partly at the system level, because the company’s skills, assets, and data accumulate. [...] Those assets then feed back into planning.”

6 Applying the Framework to Developer Discussions

Using our framework as an analytical lens, we examined public developer discussions to identify supervisory patterns that extend beyond individual instances. The framework allowed us to bring together practices across discussions and examine recurring patterns across the supervision process. Specifically, we focused on (1) what required developer involvement as more execution was delegated to agents and (2) how developers adapted their supervision in response.

We chose Reddit² as a source of developer discussions because its developer communities host naturally occurring, in-depth discussions of concrete experiences with AI coding agents [4, 35, 56]. These discussions span diverse tasks, codebases, and agent setups, providing large-scale evidence across development contexts. Hence, we collected discussions from three developer subreddits and used our framework to systematically identify and analyze supervisory practices.

6.1 Data Collection

We collected Reddit discussions and screened them to obtain a final sample of 102 threads with 12,912 comments. We describe this data collection and screening process in this section.

We selected three developer subreddits for data collection: *r/ClaudeCode*³, *r/codex*⁴, and *r/ExperiencedDevs*⁵. We chose *r/ClaudeCode* and *r/codex* because they are active communities centered on two widely used coding agents, and *r/ExperiencedDevs* because it restricts participation to developers with at least three years of experience. Together, these communities capture both agent-focused discussions and perspectives from experienced developers.

Given the rapid evolution of agentic tools and practices, we focused on a recent three-month period (May 17–August 17, 2026) to capture current practices at sufficient scale. Using Project Arctic Shift [28], we first collected the posts from the three subreddits during this period, including each post’s title, body, timestamp, and other metadata. This yielded 41,757 posts in total: 23,060 from *r/ClaudeCode*, 16,271 from *r/codex*, and 2,426 from *r/ExperiencedDevs*.

We next screened these posts to identify threads with substantial evidence of supervisory practices for qualitative analysis. We first removed posts that did not provide useful data: (1) posts with deleted content, (2) posts with no comments, and (3) posts with less relevant flairs (e.g., *Humor*, *News*; see Appendix C for details), leaving 11,471 posts.

We then used gpt-5.6-luna⁶ in two screening steps to further narrow the corpus. In the first step, the model used each post’s title and body to conservatively exclude posts unrelated to the use of AI in software development, retaining 5,041 posts. For these posts, we collected all comments to reconstruct the complete threads. In the second step, the model tagged verbatim excerpts corresponding to any of the seven supervision stages defined in Study 1, identifying 4,390 threads with at least one excerpt. The prompts used for both screening steps are provided in Appendix D.

We ranked these threads by the number of tagged excerpts and selected the top 100, including ties, for a total of 103 threads. We then manually checked each thread for relevance, excluding one that primarily promoted a product. The final sample consisted of 102 threads with 12,912 comments: 70 from *r/ClaudeCode* (9,151 comments), 26 from *r/codex* (2,631 comments), and 6 from *r/ExperiencedDevs* (1,130 comments).

6.2 Thematic Analysis

We conducted an inductive thematic analysis of the selected 102 threads following the method of Braun and Clarke [9]. We began by randomly sampling 30 threads (29.4%) to first develop an initial codebook. Three authors first divided these

²<https://www.reddit.com/>

³<https://www.reddit.com/r/ClaudeCode/>

⁴<https://www.reddit.com/r/codex/>

⁵<https://www.reddit.com/r/ExperiencedDevs/>

⁶<https://developers.openai.com/api/docs/models/gpt-5.6-luna>

sample threads and read each discussion thoroughly, tagging all relevant verbatim excerpts for the seven supervision stages. The prior LLM tagging results were visible during this pass, and we revised or removed them where they did not match our own judgment. This produced 1,114 evidence quotes: 366 for REVIEW, 230 for UPDATE ASSETS, 204 for PLAN, 127 for TEACH, 74 for WAIT, 73 for MONITOR, and 40 for MANUAL FIX.

We pooled the excerpts by each stage to preserve their context and familiarized ourselves with the data. The three authors then independently open-coded the excerpts within each stage, focusing on supervisory demands and the strategies developers used to address them. We compared our codes, merged convergent ones, discussed differences in interpretations, and consolidated them into an initial codebook. Two authors applied the initial codebook to the remaining 72 threads, discussing new or ambiguous cases and iteratively revising the codebook. One additional code emerged during this process, and no further changes were needed. Finally, we compared the resulting codes across stages and grouped them into themes that captured patterns in demands and strategies across the supervision process.

7 Findings

Applying our framework to developer discussions revealed patterns across the supervision process. We present our findings on (1) the supervisory demands developers encountered and (2) the strategies they used to address them.

7.1 Demands of Supervising AI Coding Agents

Across the seven supervisory stages, we identified three demands that spanned multiple stages: keeping agent work aligned with developer intent, enforcing quality standards beyond task completion, and maintaining understanding of agent-produced work. Each demand appeared at different points in the workflow, taking different forms as developers planned, monitored, reviewed, and corrected agent work. We describe these three demands below.

7.1.1 Keeping Agent Work Aligned with Developer Intent. A key supervisory demand was ensuring that agent work remained aligned with the developer’s intent throughout the workflow. During PLAN, developers established the goals, requirements, and boundaries that the agent was expected to follow, but agents could interpret the plan differently, change its criteria, make unintended assumptions, or expand the task scope on their own.

During MONITOR, developers had to detect when the agent’s work began to diverge from the intended plan. This divergence took several forms. Agents sometimes changed the plan itself, as one developer emphasized the need to ensure that it *“doesn’t make up criteria or change criteria as it goes.”* Others skimmed the agent’s conversation to *“spot how it deviates in an unwanted direction.”* Execution could also stall rather than progress as planned, leading one developer to set up *“monitors for when my agents get hung or stuck in place.”* Developers therefore monitored the agent during execution to catch these problems before they accumulated into larger amounts of misaligned work.

The same demand extended into REVIEW, where developers assessed alignment again after the agent had completed its work. Here, they checked both whether the implementation had become unnecessarily complex and whether the agent had introduced work beyond the intended scope. One developer described how review *“checks both things: did we make it more complicated than needed, and did the agent actually build what was in the plan instead of sneaking in extra work. If it did something outside the plan, that is a problem, even if the extra thing looks reasonable on its own.”*

When REVIEW revealed misalignment, this demand carried into TEACH, where developers redirected the agent toward the original plan. One developer explained, *“If I catch it over-engineering, I can give some push back and it makes the appropriate change.”* However, repeated correction could itself produce drift. One developer described *“scope drift over long sessions,”* where the agent *“starts building on its own previous assumptions”* across successive fixes.

7.1.2 Enforcing Quality Standards Beyond Task Completion. Even when agents followed the intended plan, developers still faced a demand to ensure that the resulting code met their quality standards. Completing the requested functionality was not sufficient, as working code could still fall short in how it was written. One developer noted that agents often produced “*lazy, sloppy, repetitive, and overly defensive code—even with solid guardrails and tests.*”

During REVIEW, developers checked aspects of code quality beyond functional correctness and carried identified issues into TEACH or MANUAL FIX for revision. One developer explained that “*part of the validation phase should include duplicate/dead code checks, among many more review items.*” When such issues were found, developers could prompt agents to improve the implementation, as one developer described, “*It really helps steering the next prompt to keep the code DRY and modular.*” Others manually made quality improvements themselves. For example, one developer “*re-write[s] most of the important comments [...] to make sure it’s easy to understand for other human reviewers.*”

7.1.3 Maintaining Understanding of Agent-Produced Work. As agents produced larger portions of the implementation, developers faced a supervisory demand to maintain enough understanding of work they had not written themselves. This involved keeping track of what the agent was doing, what had changed, and how the codebase was evolving.

During MONITOR and REVIEW, developers maintained understanding by following the agent’s work and reviewing its results. One developer explained, “*I like to read the code and talk with the model while it’s working. That way I understand what’s happening and stay engaged with it.*” Another described reading the code after execution to keep their “*mental model of how the code works*”, calling it a way to “*stay in control and to keep understanding the system that is being built.*”

The need to maintain understanding also appeared in MANUAL FIX. Developers sometimes remained directly involved in technically important parts of the implementation, particularly where broader system implications had to be understood. One developer explained, “*I still code the tricky architectural decisions myself, the stuff where you need to think through the full system implications. [...] [It] forces you to understand what the agent is actually building.*”

7.2 Strategies for Supervising AI Coding Agents

We identified three strategies through which developers organized supervisory effort across the workflow: concentrating effort in planning, distributing supervisory work between developers and agents, and externalizing supervision into reusable assets. Together, these strategies shaped where supervisory effort was concentrated, who performed it, and whether it needed to be repeated in later work. We present these strategies below.

7.2.1 Concentrating Supervisory Effort in Planning. Developers concentrated supervisory effort in the PLAN stage, investing more effort upfront to reduce the supervision required later in the workflow. Developers described well-structured plans as allowing them to shift from MONITOR to WAIT, enabling them to have agents run “*for days without interruption and come back to clean and efficient implementation of code*” and gain “*free time to manage other agents.*” They also associated stronger plans with fewer repeated TEACH cycles, as one developer noted that with a sufficiently tight architecture and testing plan, the “*whole thing can run in literally one pass, no looping needed.*” Planning also helped developers build their own understanding before implementation was delegated, with one developer noting that they got “*a very clear understanding of how the features should work, how the architecture should look, and which technical decisions needed to be made. A lot of problems were solved before any code was written.*” Below, we describe the strategies developers used when constructing plans, deciding what to include in them, and delivering those plans to agents.

Construction of the Plan. Developers often had a clear idea of what they wanted before planning, but their intent still had to be articulated with natural language in a form the agent could follow. Since the resulting plan served

directly as the agent’s instruction, developers involved agents in constructing it to expose assumptions, ambiguities, or missing details before execution. One developer brought a plan they had already drafted to the agent for stress testing, explaining that they would *“sit down and plan it out. Architect the solution to a problem. Then I put it through the wringer with AI, stress testing it and looking to see if I missed anything. Then the plan gets drafted.”* Others used dedicated planning skills that prompted the agent to interview them for missing details. One developer would *“describe feature/task and ask it to /grill-me on it,”* then *“answer questions until the skill reaches full understanding.”*

Developers also constructed plans through multiple agents, using disagreement between them to expose gaps or weak assumptions in the plan. One developer asked two models to write plans from similar prompts and then had them *“incorporate from each other’s good ideas,”* letting *“the one that did better consolidate and merge and then again the other to check it.”* Others separated plan generation from plan review. One developer had Claude prepare the plan while *“Codex does the review, and critique is messaged back to Claude,”* after which *“they loop till agreement.”*

Components of the Plan. Once the direction was settled, developers specified it through concrete constraints and representations that agents could act on. One set of components defined the boundaries of the work. Developers stated non-goals alongside goals to clarify the intended scope of a feature. One developer emphasized being *“very explicit”* about these boundaries, giving examples such as *“do not start x,” “do not rewrite y,”* and *“preserve z.”* Developers also specified which parts of the codebase the agent was allowed to modify. One described defining a *“change contract”* before prompting the agent, including *“allowed files, expected behavior, and what it must not touch.”*

Another set of components represented the intended solution more concretely. Some developers provided examples of the desired result, with one recommending that, for a new feature, developers first write *“the example you wish worked”* and then have the agent build toward it. Others supplied architecture diagrams or pseudocode to specify the intended structure. One developer described giving the model their own pseudocode to fill in, explaining that this *“ensures that its only building what I ask, and also importantly ensures that I know what the design of the codebase is.”*

Delivery of the Plan to Agents. Developers structured how plans were delivered to agents because, as one developer put it, agents were *“great at executing, but pretty bad at preserving product direction over time”* unless developers imposed structure around them. One strategy was to divide planned work into smaller execution units. One developer described breaking features down *“more than I would have done when I was manually doing it,”* reporting that this brought *“a lot more success [...] [and] seems to produce higher quality.”* Developers then paired these smaller units with explicit end conditions. One developer explained, *“Write the definition of done before it starts. One line: ‘done = this test passes and nothing outside file X changes.’ Then anything it wants to do that isn’t that is, by definition, a rabbit hole.”*

Developers also kept the plan visible throughout execution instead of relying on agents to retain it from the initial instruction. One developer required *“a detailed architecture plan”* and *“refer[red] to that plan in each step”* as implementation progressed. For longer work, developers managed context so the plan remained salient, for example by using separate branches to keep *“the context window clean enough that it doesn’t drift”* or external tools that *“re-pin your goal + context health to the bottom of the window every prompt so the model doesn’t lose the plot.”*

7.2.2 Distributing Supervisory Work. Developers addressed supervisory demands by distributing supervisory work between themselves and agents. They decided which supervisory tasks agents could perform independently, which required developer involvement, and which developers should retain entirely, covering the workflow without handling every demand directly. We describe these three arrangements below.

Delegated Supervision. Developers delegated parts of supervisory work to agents or automated mechanisms, allowing some supervisory loops to proceed without direct developer involvement. Such delegation required deciding in advance what would be watched, how detected problems should be handled, and when the corrective cycle should stop. Once configured, these arrangements allowed supervisory loops to proceed without developer involvement. In MONITOR, one developer assigned other agents to detect when an agent became “*hung or stuck in place*,” so that they could “*help the original agent out of the hole*.” In TEACH, developers set up feedback loops in which one agent reviewed another agent’s implementation and passed problems back for correction, as in an arrangement where an “*Advisor checks the implementation, calls out anything bad, back to Implementer, repeat till it’s clean*.”

REVIEW was delegated to agents most extensively, and developers were deliberate about how they configured the reviewing agents. One approach was to separate the reviewer from the implementer by model provider. One developer used “*multiple models from different providers to have them all audit each other’s work*,” explaining that each model had “*its own biases and blindspots*” and that “*getting a second opinion from a different architecture is extremely useful*.” Another approach was to separate reviews by focus, running “*one pass for security/data loss, one for test gaps, one for overengineering*,” because “*when one review tries to cover everything, it tends to produce generic advice*.”

Developers also used hooks to make parts of these supervisory loops run automatically and reliably. One developer contrasted hooks with written guidance, which could be “*read, weighed against everything else, and quietly deprioritised once context fills up*,” whereas “*a hook is code; it fires every time whether the model thinks it’s relevant or not*.” This allowed developers to embed specific supervisory actions directly into the workflow. For example, one used a stop hook that prompted an agent entering a particular area of the codebase to “*first read the system patterns for that area*.” Another used a PostToolUse hook that ran “*the narrowest relevant test right after an edit*” and fed failures back immediately, so the agent learned it had broken something “*on the same turn, before it stacks three more edits on a broken base*.”

Human-Agent Collaborative Supervision. Developers also shared supervisory work with agents, allowing agents to handle ongoing checks while retaining points for human inspection and judgment. In MONITOR, agents handled routine coordination, while developers were brought in when needed. One developer built a relay that “*passes messages between Claude and Codex*,” but “*if anything weird happens it always default freezes the relay and alerts me*.”

This collaboration was especially common in REVIEW. Developers reviewed the work after multiple rounds of agent review, saying “*agents and humans catch different types of errors*.” They also divided review so agents handled more routine checks, letting developers focus on what remained. As one explained, “*by the time I look at a change I’m reviewing something that’s already been checked for obvious mistakes, failed validations, and deviations from the task specification*.”

Developer-Retained Supervision. Developers sometimes retained supervisory work because delegating it to the agent carried its own cost. For small changes, explaining and routing the work through an agent could require more effort than making the change directly. One developer therefore made “*surgical edits for things that would take longer to type into a prompt than to just fix myself*.” Repeated correction created a similar cost, with one finding it “*mentally much less taxing to just code it up manually*” than to keep prompting the agent for the same outcome.

Developers also retained supervision to maintain their own understanding of the work. One developer described they “*read the reasoning to steer it when needed or learn more about how to properly direct it the next time I prompt*.” Another periodically rewrote “*one risky function or migration without the agent, just to keep the map in my head*.”

7.2.3 Externalizing Repeated Supervision into Assets. Developers also externalized parts of supervision into reusable assets rather than repeating the same work in each interaction. Some assets guided agents directly, while others helped

developers recover context and maintain understanding. Although creating and maintaining these assets required effort, they reduced supervisory demands that would otherwise recur later. We describe each below.

Assets for Agents. Developers turned recurring supervision into assets that reduced the need to repeat the same intervention in later work. As one developer put it, *“if I catch myself repeating the same instruction to the agent, that’s not a prompting problem, that’s a missing hook.”* One developer, after getting *“tired of writing the same review comments,”* built *“a static linter that checks for this automatically before commit.”* Another maintained a *“MISTAKES.md file where every mistake the agent makes gets documented, along with a rule in CLAUDE.md telling it to record them.”* Developers also used completed work to strengthen these assets for future supervision. One ran a retrospective at the end of each milestone, where the *“retro edits or writes skill files and meta-learnings as memory.”* Across projects, this produced *“compounding effects, essentially preventing the same kind of mistakes from happening in the future.”*

Assets for Developers. Developers used assets to reduce the effort required to maintain their own understanding of agent-produced work. Instead of reconstructing the purpose, decisions, and current state of the work each time, they kept records they could return to later. One developer recommended a feature document describing *“what this code is supposed to do without any implementation details,”* explaining that this *“helps agents and humans track the context of this work.”* Others used assets to recover their context after stepping away. One described a *“wrap-up skill that takes what you are doing and next steps and puts it in a file,”* so that on returning, it could *“remind you what’s going on.”*

8 Discussion

In this work, we developed a framework for supervising AI coding agents and applied it to developer discussions. Our findings suggest that delegating execution to agents redistributes supervisory work across the development process. Building on these findings, we discuss the framework’s applicability beyond software development, design implications for agentic systems, and how the framework can be further extended as agent autonomy increases.

8.1 Applicability of the Framework

Our framework captures supervisory functions that may extend beyond software development to other forms of delegated agentic work. Software development has been at the forefront of agentic delegation. Similar forms of delegation have now started to emerge in knowledge and creative work, where agents support tasks such as writing, research, data analysis, and design [23, 24, 51]. A particularly relevant example is computer-use agents, which can carry out tasks across browsers, documents, spreadsheets, and communication tools with limited human involvement [3, 49, 52]. As users delegate such work, they may need to specify goals and constraints, oversee execution, evaluate outcomes, and intervene when necessary. Thus, our framework may generalize beyond software development, providing a common structure for analyzing these supervisory functions across domains.

Applying the framework in these settings would require adapting each supervisory stage to the characteristics of the delegated work. In many knowledge and creative tasks, goals and quality criteria may be less precisely specified, and outputs cannot always be verified through deterministic checks. For example, in design tasks, PLAN may involve specifying target users, interaction goals, and visual constraints, while REVIEW may rely more on human judgment of usability, coherence, and aesthetic quality. Applying the framework across such domains could reveal how supervisory effort is distributed differently across types of work and where new demands for human involvement arise.

8.2 Design Implications for Agentic Systems

We present design implications for supporting developer supervision of AI coding agents.

8.2.1 Developing AI Coding Agents. Our findings suggest several directions for developing AI coding agents that require less supervisory effort from developers. First, developers of AI coding agents should prioritize preserving developer intent throughout execution. Developers faced supervisory demands when agents deviated from intended goals, constraints, or scope (§7.1.1). Thus, agents should be designed to consistently follow the plans and constraints provided by developers, reducing the need for repeated monitoring and redirection. For example, methods for maintaining goals and constraints over longer interactions [41] or detecting when an agent’s planned actions conflict with previously specified intent [6] could help prevent such deviations before developer intervention becomes necessary.

Second, developers of AI coding agents should account for quality expectations beyond functional correctness. Even when functionality was correct, developers had to address implementations they considered repetitive, overly defensive, or difficult to maintain (§7.1.2). Hence, agents should be designed to reflect developer- and project-specific preferences in how code is produced. For instance, preference-alignment methods [59, 73] could be used to align agent outputs with preferred coding conventions, reducing the need to repeatedly correct the same quality issues.

8.2.2 Supporting Developer Supervision. Our findings also suggest several ways systems can be designed to better support developers in supervising AI coding agents. First, systems can better support developers in the planning stage prior to task delegation. Developers invested substantial effort in constructing plans that clarified goals, boundaries, and success criteria and reduced supervision later in the workflow (§7.2.1). Systems could help developers articulate these elements, identify missing or ambiguous requirements, and keep plans accessible throughout execution [22, 34, 77].

Second, systems can be designed to reduce the effort of setting up and maintaining delegated supervision. Developers reduced supervisory work by delegating MONITOR and REVIEW to other agents (§7.2.2) and using reusable assets such as instruction files, tests, and hooks for repeated supervision (§7.2.3). However, these arrangements required developers to configure and maintain multiple agents and assets. Systems could help developers manage these forms of delegated supervision together, making it easier to inspect, update, and reuse how supervision is set up for a task [42, 53, 74].

Finally, systems can be designed to help developers maintain an understanding of work delegated to agents. Developers actively followed and revisited agent-generated code to maintain a mental model of the codebase over time (§7.1.3). Systems could support this by highlighting important changes, connecting them to the agent’s rationale, and summarizing the context needed for developers to understand what changed and why [30, 32, 82].

8.3 Further Development of the Framework

Although our framework captures a broad range of supervisory activities in agentic software development, the structure of supervision may change as agent autonomy increases. Prior work suggests that as AI agents become more autonomous, users may shift from actively directing their work toward approving or even observing it [14, 21, 85]. Such changes may redistribute human involvement across the supervisory process, making some stages less prominent while increasing the importance of others. For example, practitioners increasingly debate whether every agent-generated change requires human review, with some arguing for human review only for high-risk or consequential changes [38, 72]. Hence, the framework can be further developed to reflect these changes as new forms of human-agent supervision emerge.

The framework could also be developed to incorporate evidence about the effectiveness of different supervisory practices. Empirical studies could examine how the allocation of supervisory effort across stages relates to development

outcomes. For example, they could test whether greater effort in *PLAN* reduces later correction, delegated *REVIEW* improves code quality, or different supervisory workflows affect developer time and errors [37]. Such evidence could add an evaluative layer to the framework, indicating which forms of supervision are more effective under what conditions.

Finally, the framework could be extended to incorporate the effects of supervision on developers. As implementation is increasingly delegated to agents, different supervisory arrangements may shape developers' agency [58, 65], reliance on AI [27, 31, 33], self-efficacy and authorship [55, 66], and ability to maintain skills and understanding over time [43, 47]. The framework could be extended to connect supervisory arrangements with their longer-term effects on developers.

9 Limitations and Future Work

We acknowledge several limitations of our study and suggest potential future work.

First, our study for framework development captured only a limited window of participants' development work. Although we asked participants to bring tasks that would allow us to observe an end-to-end supervisory workflow within 40 minutes, real-world development projects often span longer periods and multiple sessions beyond what we could capture. The subsequent workflow design activity helped participants extend their diagrams beyond the observed task by considering other tasks and developers, but this still relied on retrospective reflection. Future work could examine longer-term supervision through longitudinal observations [26, 67, 78] or interaction traces [75].

Second, participants' observed behavior and resulting workflow diagrams may have been affected by the study setting. Participants completed their tasks while being observed and thinking aloud, which may have altered how they worked. Moreover, although we introduced Sheridan's framework to help participants reflect on how their workflows related to it and emphasized that they need not follow its structure, prior exposure may still have anchored how they segmented or represented their supervision. Future work could complement our approach with less intrusive observations and workflow elicitation without prior exposure to a theoretical framework.

10 Conclusion

In this work, we developed a framework for supervising AI coding agents, reconfiguring Sheridan's framework of *human supervisory control*. We then applied our framework to developer discussions on agentic AI in software development and found that supervisory demands extend across stages. Developers manage such demands by shifting effort across the workflow, delegating parts of supervision, and creating reusable assets. These findings highlight that increasing autonomy of agents changes where and how human involvement is needed. Our framework provides a lens for understanding and supporting human supervision as agentic software development continues to evolve.

References

- [1] Mamdouh Alenezi. 2026. Rethinking Software Engineering for Agentic AI Systems. arXiv:2604.10599 [cs.SE] <https://arxiv.org/abs/2604.10599>
- [2] Anthropic. 2025. *Claude Code Documentation*. <https://code.claude.com/docs> Accessed: 2026-09-11.
- [3] Anthropic. 2026. *Claude Fable and Mythos 5.1*. Anthropic. <https://www.anthropic.com/claude-fable-and-mythos-5-1> Accessed: 2026-09-10.
- [4] Sebastian Baltes, Marc Cheong, and Christoph Treude. 2026. "An Endless Stream of AI Slop": How Developers Discuss the Burden of AI-Assisted Software Development. arXiv:2603.27249 [cs.SE] <https://arxiv.org/abs/2603.27249>
- [5] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 78 (April 2023), 27 pages. doi:10.1145/3586030
- [6] Fraol Batole, Foutse Khomh, and Hridesh Rajan. 2026. RefineAct: Automatic Runtime Verification of LLM Agent Actions. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering* (Munich, Germany) (ASE '26). Association for Computing Machinery, New York, NY, USA, 12 pages.
- [7] Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. 2026. SWE-chat: Coding Agent Interactions From Real Users in the Wild. arXiv:2604.20779 [cs.AI] <https://arxiv.org/abs/2604.20779>

- [8] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. arXiv:2507.09089 [cs.AI] <https://arxiv.org/abs/2507.09089>
- [9] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa
- [10] Elena Cavallin and Simone Spagnol. 2026. When Designers Sweat: Behavioral Traces of GenAI Co-Creation. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 154, 17 pages. doi:10.1145/3772318.3791776
- [11] Chaoran Chen, Zhiping Zhang, Zeya Chen, Eryue Xu, Yinuo Yang, Ibrahim Khalilov, Simret A Gebreegziabher, Yanfang Ye, Ziang Xiao, Yaxing Yao, Tianshi Li, and Toby Jia-Jun Li. 2026. Comparing Human Oversight Strategies for Computer-Use Agents. arXiv:2604.04918 [cs.HC] <https://arxiv.org/abs/2604.04918>
- [12] Valerie Chen, Ameet Talwalkar, Robert Brennan, and Graham Neubig. 2026. Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 122, 19 pages. doi:10.1145/3772318.3790850
- [13] Ria Cheruvu. 2025. *Human-in/on-the-Loop Design for Human Controllability*. Springer Nature Singapore, Singapore, 1–47. doi:10.1007/978-981-97-8440-0_75-1
- [14] Rudrajit Choudhuri, Christian Bird, Carmen Badea, Marco Gerosa, and Anita Sarma. 2026. You Shall Not Pass! Where and Why Developers Draw The Line on AI Autonomy. arXiv:2607.00533 [cs.HC] <https://arxiv.org/abs/2607.00533>
- [15] Shipi Dhanorkar, Samir Passi, and Mihaela Vorvoreanu. 2026. *Human oversight of agentic systems in practice: Examining the oversight work, challenges, and heuristics of developers using software agents*. Association for Computing Machinery, New York, NY, USA, 6438–6465. <https://doi.org/10.1145/3805689.3812402>
- [16] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. 2025. A Survey on Code Generation with LLM-based Agents. arXiv:2508.00083 [cs.SE] <https://arxiv.org/abs/2508.00083>
- [17] Satu Elo and Helvi Kyngäs. 2008. The qualitative content analysis process. *Journal of Advanced Nursing* 62, 1 (2008), 107–115. doi:10.1111/j.1365-2648.2007.04569.x
- [18] Guangrui Fan, Dandan Liu, Lihu Pan, and Rui Zhang. 2026. When Help Hurts: Verification Load and Fatigue with AI Coding Assistants. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 733, 25 pages. doi:10.1145/3772318.3791176
- [19] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2026. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook – a Grey Literature Review. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26)*. Association for Computing Machinery, New York, NY, USA, 212–223. doi:10.1145/3786583.3786866
- [20] Dana Feng, Bhada Yun, and April Yi Wang. 2026. From Junior to Senior: Allocating Agency and Navigating Professional Growth in Agentic AI-Mediated Software Engineering. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 596, 24 pages. doi:10.1145/3772318.3791642
- [21] Kevin J. Feng, David W. McDonald, and Amy X. Zhang. 2025. *Levels of Autonomy for AI Agents*. Technical Report 25-15. Knight First Amendment Institute at Columbia University. <https://knightcolumbia.org/content/levels-of-autonomy-for-ai-agents-1>
- [22] K. J. Kevin Feng, Kevin Pu, Matt Latzke, Tal August, Pao Siangliulue, Jonathan Bragg, Daniel S Weld, Amy X. Zhang, and Joseph Chee Chang. 2026. Cocoa: Co-Planning and Co-Execution with AI Agents. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 16, 23 pages. doi:10.1145/3772318.3791673
- [23] Figma. 2026. *The Figma Agent Is Here*. Figma Blog. <https://www.figma.com/blog/the-figma-agent-is-here/> Accessed: 2026-09-10.
- [24] Google Cloud. 2026. *Introducing Gemini Enterprise Agent Platform, powering the next wave of agents*. Google Cloud Blog. <https://cloud.google.com/blog/products/ai-machine-learning/introducing-gemini-enterprise-agent-platform> Accessed: 2026-09-10.
- [25] Jiale Guo, Suizhi Huang, Mei Li, Dong Huang, Xingsheng Chen, Regina Zhang, Zhijiang Guo, Han Yu, Siu-Ming Yiu, Pietro Lio, and Kwok-Yan Lam. 2025. A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System. arXiv:2510.09721 [cs.SE] <https://arxiv.org/abs/2510.09721>
- [26] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2026. *Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects*. Association for Computing Machinery, New York, NY, USA, 181–193. <https://doi.org/10.1145/3793302.3793349>
- [27] Keyu He, Qianou Ma, Valerie Chen, Wayne Chi, and Tongshuang Wu. 2026. RECAP: An End-to-End Platform for Capturing, Replaying, and Analyzing AI-Assisted Programming Interactions. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Greg Durrett and Ping Jian (Eds.). Association for Computational Linguistics, San Diego, California, United States, 692–701. doi:10.18653/v1/2026.acl-demo.68
- [28] Arthur Heitmann. 2026. Project Arctic Shift. <https://arctic-shift.photon-reddit.com/> Accessed: 2026-09-03.
- [29] Ruanqianqian Huang, Avery Reyna, Sorin Lerner, Haijun Xia, and Brian Hempel. 2025. Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025. arXiv:2512.14012 [cs.SE] <https://arxiv.org/abs/2512.14012>
- [30] Robert Hutter and Michael Pradel. 2026. AgentStepper: Interactive Debugging of Software Development Agents. arXiv:2602.06593 [cs.SE] <https://arxiv.org/abs/2602.06593>

- [31] Lujain Ibrahim, Katherine M. Collins, Sunnie S. Y. Kim, Anka Reuel, Max Lamparth, Kevin Feng, Lama Ahmad, Prajna Soni, Alia El Kattan, Merlin Stein, Siddharth Swaroop, Vishakh Padmakumar, Ilia Sucholutsky, Andrew Strait, Diyi Yang, Q. Vera Liao, and Umang Bhatt. 2026. Measuring and mitigating overreliance to build human-compatible AI. *arXiv:2509.08010 [cs.CY]* <https://arxiv.org/abs/2509.08010>
- [32] Majeed Kazemitabaar, Jack Williams, Ian Drosos, Tovi Grossman, Austin Zachary Henley, Carina Negreanu, and Advait Sarkar. 2024. Improving Steering and Verification in AI-Assisted Data Analysis with Interactive Task Decomposition. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) (*UIST '24*). Association for Computing Machinery, New York, NY, USA, Article 92, 19 pages. doi:10.1145/3654777.3676345
- [33] Sunnie S. Y. Kim, Jennifer Wortman Vaughan, Q. Vera Liao, Tania Lombrozo, and Olga Russakovsky. 2025. Fostering Appropriate Reliance on Large Language Models: The Role of Explanations, Sources, and Inconsistencies. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 420, 19 pages. doi:10.1145/3706598.3714020
- [34] Tae Soo Kim, Yoonjoo Lee, Jamin Shin, Young-Ho Kim, and Juho Kim. 2024. EvalLM: Interactive Evaluation of Large Language Model Prompts on User-Defined Criteria. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (*CHI '24*). Association for Computing Machinery, New York, NY, USA, Article 306, 21 pages. doi:10.1145/3613904.3642216
- [35] Jan H. Klemmer, Stefan Albert Horstmann, Nikhil Patnaik, Cordelia Ludden, Cordell Burton, Carson Powers, Fabio Massacci, Akond Rahman, Daniel Votipka, Heather Richter Lipford, Awais Rashid, Alena Naiakshina, and Sascha Fahl. 2024. Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) (*CCS '24*). Association for Computing Machinery, New York, NY, USA, 2726–2740. doi:10.1145/3658644.3690283
- [36] Amy J. Ko, Thomas D. LaToza, and Margaret M. Burnett. 2015. A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Softw. Engg.* 20, 1 (Feb. 2015), 110–141. doi:10.1007/s10664-013-9279-3
- [37] Vivian Lai, Samuel Carton, Rajat Bhatnagar, Q. Vera Liao, Yunfeng Zhang, and Chenhao Tan. 2022. Human-AI Collaboration via Conditional Delegation: A Case Study of Content Moderation. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (*CHI '22*). Association for Computing Machinery, New York, NY, USA, Article 54, 18 pages. doi:10.1145/3491102.3501999
- [38] Rachel Laycock. 2026. Maybe We Shouldn't Be Reviewing All This Code. <https://martinfowler.com/rachels-ramblings/code-review.html>. Blog post, martinfowler.com.
- [39] Florian Lehmann, Krystsina Shauchenka, and Daniel Buschek. 2026. Collaborative Document Editing with Multiple Users and AI Agents. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 58, 27 pages. doi:10.1145/3772318.3790648
- [40] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Trans. Softw. Eng. Methodol.* (March 2026). doi:10.1145/3796507 Just Accepted.
- [41] Shukai Liu, Bo Jiang, Jian Yang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. 2026. Context as a Tool: Context Management for Long-Horizon SWE-Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (Eds.). Association for Computational Linguistics, San Diego, California, United States, 20604–20617. doi:10.18653/v1/2026.findings-acl.1032
- [42] Yang Liu, Jinxuan Cai, Yishen Li, Qi Meng, Zedi Liu, Xin Li, Chen Qian, Chuan Shi, and Cheng Yang. 2026. MASFactory: A Graph-centric Framework for Orchestrating LLM-Based Multi-Agent Systems with Vibe Graphing. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Greg Durrett and Ping Jian (Eds.). Association for Computational Linguistics, San Diego, California, United States, 357–366. doi:10.18653/v1/2026.acl-demo.35
- [43] Qianou Ma, Kenneth R Koedinger, and Tongshuang Wu. 2026. Not Everyone Wins with LLMs: Behavioral Patterns and Pedagogical Implications for AI Literacy in Programmatic Data Science. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 139, 22 pages. doi:10.1145/3772318.3791283
- [44] Qianou Ma, Weirui Peng, Chenyang Yang, Hua Shen, Ken Koedinger, and Tongshuang Wu. 2025. What Should We Engineer in Prompts? Training Humans in Requirement-Driven LLM Use. *ACM Trans. Comput.-Hum. Interact.* 32, 4, Article 41 (Aug. 2025), 27 pages. doi:10.1145/3731756
- [45] Qianou Ma, Tongshuang Wu, and Kenneth Koedinger. 2023. Is AI the better programming partner? Human-Human Pair Programming vs. Human-AI pAIr Programming. *arXiv:2306.05153 [cs.HC]* <https://arxiv.org/abs/2306.05153>
- [46] Sebastian Maier, Manuel Schneider, and Stefan Feuerriegel. 2026. Partnering with Generative AI: Experimental Evaluation of Model-Led and Human-Led Interaction in Human-AI Co-Creation. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 1260, 28 pages. doi:10.1145/3772318.3791185
- [47] Margaret Mitchell, Avijit Ghosh, and Samir Passi. 2026. AI Agents Push Humans Out of the Loop. *arXiv:2608.23642 [cs.AI]* <https://arxiv.org/abs/2608.23642>
- [48] Hussein Mozannar, Gagan Bansal, Adam Fournay, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (*CHI '24*). Association for Computing Machinery, New York, NY, USA, Article 142, 16 pages. doi:10.1145/3613904.3641936
- [49] OpenAI. 2025. *Introducing Operator*. OpenAI Blog. <https://openai.com/index/introducing-operator/> Accessed: 2026-09-10.
- [50] OpenAI. 2025. *OpenAI Codex*. <https://openai.com/codex/> Accessed: 2026-09-11.
- [51] OpenAI. 2026. *ChatGPT Is Now a Partner for Your Most Ambitious Work*. OpenAI. <https://openai.com/index/chatgpt-for-your-most-ambitious-work/> Accessed: 2026-09-10.
- [52] OpenAI. 2026. *GPT-6 Astra*. OpenAI. <https://openai.com/index/gpt-6-astra/> Accessed: 2026-09-10.

- [53] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 2026. Natural-Language Agent Harnesses. arXiv:2603.25723 [cs.CL] <https://arxiv.org/abs/2603.25723>
- [54] R. Parasuraman, T.B. Sheridan, and C.D. Wickens. 2000. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30, 3 (May 2000), 286–297. doi:10.1109/3468.844354
- [55] Yeon Su Park, Nadia Azzahra Putri Arvi, Seoyoung Kim, and Juho Kim. 2026. Authorship Drift: How Self-Efficacy and Trust Evolve During LLM-Assisted Writing. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 1549, 18 pages. doi:10.1145/3772318.3790276
- [56] Veronica Pimenova, Sarah Fakhoury, Christian Bird, Margaret-Anne Storey, and Madeline Endres. 2026. Good Vibrations? A Qualitative Study of Co-Creation, Communication, Flow, and Trust in Vibe Coding. arXiv:2509.12491 [cs.SE] <https://arxiv.org/abs/2509.12491>
- [57] James Prather, Brent N. Reeves, Paul Denny, Brett A. Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. “It’s Weird That it Knows What I Want”: Usability and Interactions with Copilot for Novice Programmers. *ACM Trans. Comput.-Hum. Interact.* 31, 1, Article 4 (Nov. 2023), 31 pages. doi:10.1145/3617367
- [58] Kevin Pu, Daniel Lazaro, Ian Arawjo, Haijun Xia, Ziang Xiao, Toví Grossman, and Yan Chen. 2025. Assistance or Disruption? Exploring and Evaluating the Design and Trade-offs of Proactive AI Programming Support. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 152, 21 pages. doi:10.1145/3706598.3713357
- [59] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 53728–53741. doi:10.52202/075280-2338
- [60] Partha Pratim Ray. 2025. A Review on Vibe Coding: Fundamentals, State-of-the-art, Challenges and Future Directions. *TechRxiv* 2025, 0509 (2025). doi:10.36227/techrxiv.174681482.27435614/v1
- [61] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic Much? Adoption of Coding Agents on GitHub. *ACM Trans. Softw. Eng. Methodol.* (June 2026). doi:10.1145/3822180 Just Accepted.
- [62] Italo Santos, Cleyton Magalhaes, and Ronnie De Souza Santos. 2025. Model-Assisted and Human-Guided: Perceptions and Practices of Software Professionals Using LLMs for Coding. In *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. 105–112. doi:10.1109/AIware69974.2025.00019
- [63] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. Vibe Coding vs. Agentic Coding: Fundamentals and Practical Implications of Agentic AI. arXiv:2505.19443 [cs.SE] <https://arxiv.org/abs/2505.19443>
- [64] Advait Sarkar, Carina Negreanu, Ben Zorn, Sruti Srinivasa Ragavan, Christian Pölitz, and Andrew D. Gordon. 2022. What is it like to program with artificial intelligence?. In *Proceedings of the 33rd Annual Workshop of the Psychology of Programming Interest Group, PPiG 2022, The Open University, Milton Keynes, UK & Online, September 5-9, 2022*, Simon Holland, Marian Petre, Luke Church, and Mariana Marasoiu (Eds.). Psychology of Programming Interest Group, 127–153. <https://ppig.org/papers/2022-ppig-33rd-sarkar/>
- [65] Ruben Schlonsak, Hans-Christian Jetter, and Denys J.C. Matthies. 2026. The Cost of Convenience: How Delegation and Decision Horizons Erode Human Agency when overusing LLM Systems. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*. Association for Computing Machinery, New York, NY, USA, Article 650, 6 pages. doi:10.1145/3772363.3798760
- [66] Jwawon Seo, Elmira Deldari, and Helena M. Mentis. 2026. Whose Code Is It? How AI Autonomy Reshapes Ownership, Responsibility, and Disclosure in AI-Assisted Programming. In *Proceedings of the 31st International Conference on Intelligent User Interfaces (IUI '26)*. Association for Computing Machinery, New York, NY, USA, 393–425. doi:10.1145/3742413.3789121
- [67] Agnia Sergeyuk, Eric Huang, Dariia Karaeva, Anastasiia Serova, Yaroslav Golubev, and Iftekhhar Ahmed. 2026. Evolving with AI: A Longitudinal Analysis of Developer Logs. arXiv:2601.10258 [cs.SE] <https://arxiv.org/abs/2601.10258>
- [68] Thomas B. Sheridan. 1976. *Toward a General Model of Supervisory Control*. Springer US, Boston, MA, 271–281. doi:10.1007/978-1-4684-2523-9_22
- [69] Thomas B. Sheridan. 1992. *Telerobotics, automation, and human supervisory control*. MIT Press, Cambridge, MA, USA.
- [70] Thomas B. Sheridan. 2012. *Human Supervisory Control*. John Wiley & Sons, Ltd, Chapter 34, 990–1015. doi:10.1002/9781118131350.ch34
- [71] Pradyumna Shome, Sashreek Krishnan, and Sauvik Das. 2026. Why Johnny Can’t Use Agents: Industry Aspirations vs. User Realities with AI Agents. In *Proceedings of the ACM Conference on AI and Agentic Systems (CAIS '26)*. Association for Computing Machinery, New York, NY, USA, 597–619. doi:10.1145/3786335.3813140
- [72] Tanjal Shukla, Kevin Feng, Leijie Wang, Mohammad Rostami, and Amy Zhang. 2026. Hedwig: Dynamic Autonomy for Coding Agents Under Local Oversight. In *Proceedings of the ACM Conference on AI and Agentic Systems (CAIS '26)*. Association for Computing Machinery, New York, NY, USA, 1293–1299. doi:10.1145/3786335.3813223
- [73] Sanjeevan Sivapiran and Gias Uddin. 2026. Reward-Free Code Alignment from Pretrained or Fine-Tuned LLM: Unpacking the Trade-offs for Code Generation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE116 (June 2026), 24 pages. doi:10.1145/3808123
- [74] Boyin Tan, Xiaowei Huang, and Youcheng Sun. 2026. Skill Coverage: A Test Adequacy Metric for Agent Skills. arXiv:2606.20659 [cs.AI] <https://arxiv.org/abs/2606.20659>
- [75] Ningzhi Tang, Chaoran Chen, Zihan Fang, Gelei Xu, Maria Dhakal, Yiyu Shi, Collin McMillan, Yu Huang, and Toby Jia-Jun Li. 2026. Programming by Chat: A Large-Scale Behavioral Analysis of 11,579 Real-World AI-Assisted IDE Sessions. arXiv:2604.00436 [cs.SE] <https://arxiv.org/abs/2604.00436>
- [76] Jonathan Ullrich, Matthias Koch, and Andreas Vogelsang. 2025. From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering. In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*. 257–266. doi:10.1109/RE63999.2025.00032

- [77] Priyan Vaithilingam, Munyeong Kim, Frida-Cecilia Acosta-Parenteau, Daniel Lee, Amine Mhedhbi, Elena L. Glassman, and Ian Arawjo. 2025. Semantic Commit: Helping Users Update Intent Specifications for AI Memory at Scale. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. Association for Computing Machinery, New York, NY, USA, Article 137, 18 pages. doi:10.1145/3746059.3747778
- [78] Annie Vella and Kelly Blincoe. 2026. The Impact of AI Coding Assistants on Software Engineering: A Longitudinal Study. arXiv:2605.23135 [cs.SE] <https://arxiv.org/abs/2605.23135>
- [79] Yun Wang and Yan Lu. 2025. Interaction, Process, Infrastructure: A Unified Framework for Human-Agent Collaboration. arXiv:2506.11718 [cs.HC] <https://arxiv.org/abs/2506.11718>
- [80] Yinan Wu, Ze Shi Li, Kathryn Thomasset Stolee, and Bowen Xu. 2026. How Do Developers Interact with AI? An Exploratory Study on Modeling Developer Programming Behavior. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE113 (June 2026), 24 pages. doi:10.1145/3808120
- [81] Xiaotong (Tone) Xu, Arina Konnova, Bianca Gao, Cindy Peng, Dave Vo, and Steven P. Dow. 2025. Productive vs. Reflective: How Different Ways of Integrating AI into Design Workflows Affect Cognition and Motivation. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 24, 15 pages. doi:10.1145/3706598.3713649
- [82] Litao Yan, Alyssa Hwang, Zhiyuan Wu, and Andrew Head. 2024. Ivie: Lightweight Anchored Explanations of Just-Generated Code. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 140, 15 pages. doi:10.1145/3613904.3642239
- [83] Bhada Yun, Dana Feng, Ace S. Chen, Afshin Nikzad, and Niloufar Salehi. 2025. Generative AI in Knowledge Work: Design Implications for Data Navigation and Decision-Making. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 634, 19 pages. doi:10.1145/3706598.3713337
- [84] J.D. Zamfirescu-Pereira, Eunice Jun, Michael Terry, Qian Yang, and Bjoern Hartmann. 2025. Beyond Code Generation: LLM-supported Exploration of the Program Design Space. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 153, 17 pages. doi:10.1145/3706598.3714154
- [85] Jieyu Zhou, Aryan Roy, Sneha Gupta, Daniel Weitekamp, and Christopher J. MacLellan. 2026. When Should Users Check? Modeling Confirmation Frequency in Multi-Step Agentic AI Tasks. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 1649, 20 pages. doi:10.1145/3772318.3790655

A Pre-Task Interview Questions

We list the full interview questions for the study below.

A.1 Participant Background

- Could you briefly introduce yourself and your current role?
- How much experience do you have with software development or programming?
- When did you start using AI coding agents?
- Could you briefly describe the AI coding tools or agent setup you usually use?
- When you delegate work to AI coding agents, what level of granularity do you usually use?
- How do you usually review or verify the outputs generated by AI coding agents?
- In your current company, team, lab, open-source project, or other work context, are there any explicit or implicit rules, norms, or expectations around using AI coding tools?
- If so, how do they affect the way you use these tools in practice?
- Compared to your colleagues or peers, in what ways do you feel your use of AI coding tools is similar or different?

A.2 Prepared Task

- Could you briefly describe the task you prepared for today?
- What would you like to accomplish within the 40-minute coding session?

B Example Supervision Workflow Diagrams

We present example supervision workflow diagrams drawn by participants in our interviews, illustrating the diversity of stages and loop structures observed. The diagrams shown here are translated to English from the original version.

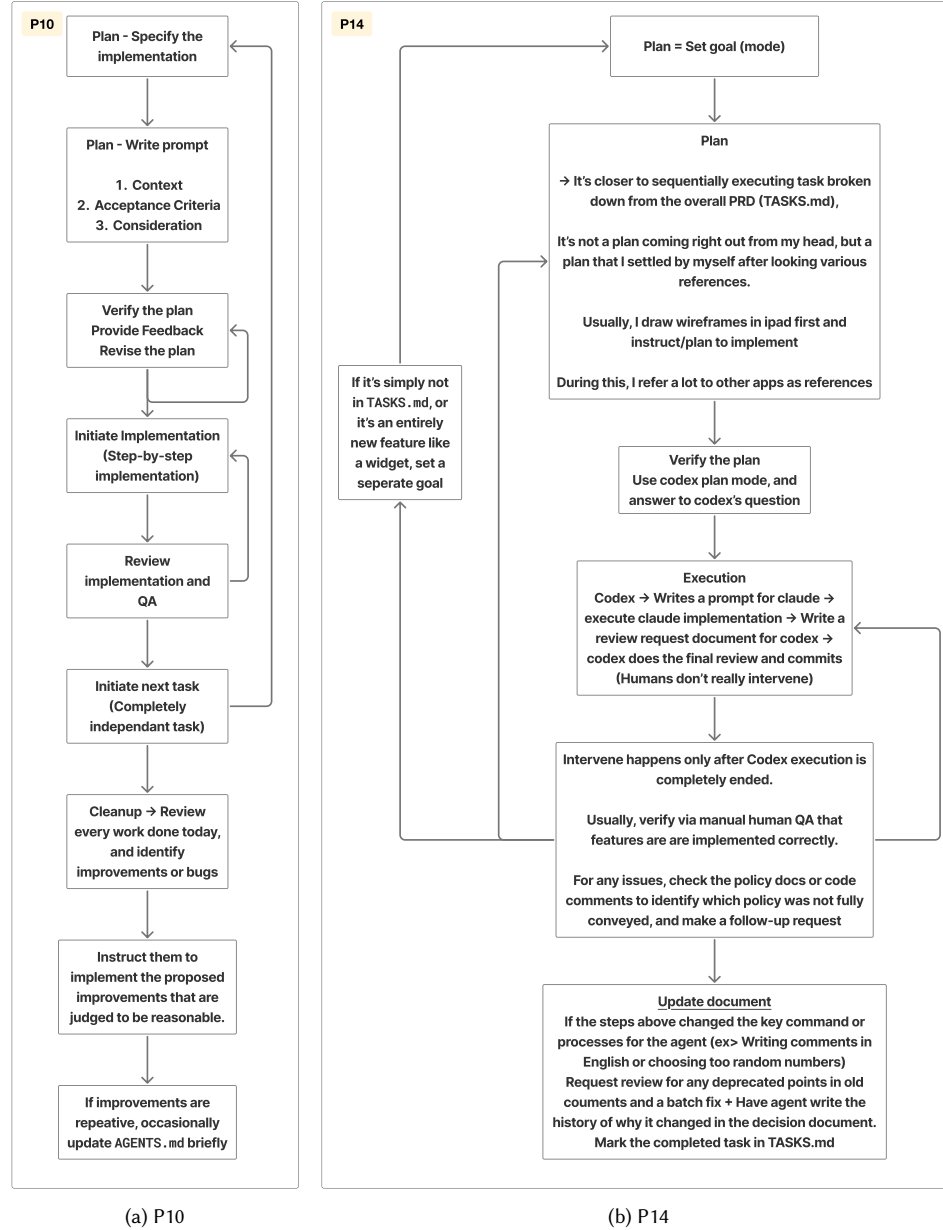


Fig. 4. Example supervision workflow diagrams from participant interviews, showing diverse stages and loop structures.

C Reddit Flair Filtering

On Reddit, post *flairs* are labels used to categorize posts within a subreddit. We used these flairs to exclude posts unlikely to contain relevant discussions before LLM-based screening. Since each subreddit uses its own set of flairs, we applied subreddit-specific filtering criteria. The flairs excluded for each subreddit are listed below.

r/ClaudeCode. We retained posts with the following flairs: *Discussion, Question, Help Needed, Help/Question, Resource, Tutorial / Guide*, and *Tips & Workflows*. On the other hand, we excluded posts with the following flairs: *Showcase, Humor, Bug Report, Built with Claude, Meta, Solved, Bug / Issue, Rant*, or *News/Updates*, as well as variants of *Megathread, Weekly Showcase*, and *Community Update*. This filtering step retained 7,232 of 10,427 posts.

r/codex. We retained posts with the following flairs: *Question, Complaint, Commentary, Suggestion, Workaround*, or *Instruction*. On the other hand, we excluded posts with the following flairs: *Showcase, Bug, Praise, Limits, News, Other, Comparison, Humor, Reset, Megathread*, or *MOD*. This filtering step retained 4,054 of 7,284 posts.

r/ExperiencedDevs. We retained posts with the following flairs: *Career/Workplace, AI/LLM*, and *Technical question*. We excluded posts labeled *Meta* or *Big Tech*, as well as posts without a flair. This filtering step retained 185 of 211 posts.

D LLM Prompts Used

We present the prompts used for the two LLM-assisted screening steps in our data collection of Reddit threads as explained in §6.1: (1) screening posts for relevance and (2) initial supervision stage tagging.

D.1 Prompt for Screening Posts for Relevance

Prompt

You are screening posts for a study of how developers supervise AI coding tools and agents during software development.

Keep a post if it meaningfully discusses how developers work with, manage, or respond to AI in software-development work. This includes concrete practices, experiences, problems, failures, decisions, role changes, or advice related to AI-assisted development.

Exclude posts that are only about:

- Model releases, benchmarks, or general performance comparisons
- Pricing, subscriptions, tokens, rate limits, or usage quotas
- Product availability, outages, account issues, or other service-related problems
- General AI, career, or future-of-programming discussion with no concrete connection to AI-assisted development work
- Building, announcing, or promoting a tool or AI system without substantively describing a relevant development practice, workflow, problem, or experience
- Choosing between tools, plans, or editors without connecting the choice to concrete development tasks, workflows, or ways of working
- Memes, jokes, advertisements, or unrelated technical support

A post may still qualify if it promotes or introduces a tool, as long as it substantively describes a relevant AI-assisted development practice, workflow, problem, or experience.

A title-only or very brief post should be kept only when the available text itself clearly describes a relevant practice, problem, or experience.

A post does not need to describe a complete workflow. A concrete discussion of even one relevant practice, experience, problem, failure, decision, or role change is sufficient.

When uncertain, keep the post.

Return only JSON:

```
{
  "keep": true,
  "confidence": "high | mid | low",
  "reason": "Brief reason"
}
```

Title:
{{TITLE}}

Body:
{{BODY}}

D.2 Prompt for Initial Supervision Stage Tagging

Prompt

You are analyzing a complete Reddit discussion thread about AI-assisted software development using the supervisory workflow framework below.

Identify every supervisory stage that is meaningfully discussed in the thread and extract all supporting evidence for each detected stage.

Read the entire thread before making any decisions.

GENERAL PRINCIPLE

The stages represent supervisory functions in AI-assisted software-development work, not necessarily actions performed directly by the developer.

A supervisory function may be performed by the developer or deliberately delegated to another

agent, tool, or automated mechanism. Tag the stage when the developer intentionally uses that function to guide, oversee, coordinate, or evaluate AI-assisted work.

Do not tag a stage merely because an AI agent autonomously performs a similar action while completing its task.

CODING RULES

- Tag a stage only when a concrete supervisory practice, experience, problem, failure, decision, or recommendation is explicitly described or clearly supported by the text.
- The evidence must concern supervising AI-assisted software-development work. General product-development practices, human-team management, or unrelated uses of AI do not count.
- General statements about AI capabilities, model quality, usefulness, reliability, or what someone could hypothetically do are not sufficient by themselves.
- Advice or recommendations may count when they describe a concrete supervisory practice, not merely a vague suggestion.
- A thread may contain multiple stages, and different participants may provide evidence for different stages. Do not combine statements from different participants into one continuous workflow.
- Do not force a stage to appear when the evidence is ambiguous. A thread may contain no supervisory stage at all.
- For each detected stage, include every independently stated example or discussion that provides meaningful evidence for that stage, even when similar practices are discussed by different participants or in different parts of the thread. Avoid duplicate or overlapping excerpts that capture the same statement from the same source.
- Each excerpt must be a contiguous, verbatim quote that includes enough context to understand why it supports the stage. Prefer complete sentences and avoid unnecessarily truncating the evidence. Do not paraphrase, combine, correct, or normalize the text. Copy `source_type`, `source_id`, and `author` exactly as supplied.

STAGES

- Plan: Structuring how an AI-assisted task should be carried out before an execution cycle by defining or refining requirements, constraints, task boundaries, success criteria, decomposition, or an intended approach. This may include using an agent to help develop or refine the plan. Simply naming a planning step, assigning a task, describing the current task, or giving general work advice does not count.
- Monitor: Observing or checking an AI agent while a specific execution cycle is still ongoing to understand its progress, actions, tool use, intermediate state, errors, or goal drift. Merely knowing that an agent is running, describing general involvement, or tracking tokens, cost, latency, context usage, or other resource metrics does not count.
- Wait: Allowing AI work to continue while the developer steps away, switches to another task, or works with another agent in parallel. The developer must leave the AI working without active intervention for some period. An agent waiting for developer input or approval, ordinary response latency, notifications,

or remote-access functionality alone do not count.

- Review: Evaluating AI-assisted work against the developer’s goals, requirements, or quality criteria before accepting it or proceeding with subsequent work. Review may be performed directly by the developer or deliberately delegated to another agent, tests, static analysis, or other verification mechanisms.
- Teach: Providing new information or direction after execution has begun in order to change the AI’s subsequent work. This includes correcting misunderstandings, adding context, redirecting the AI, or asking it to revise, retry, or use another approach. Merely asking how feedback could be provided does not count.
- Manual Fix: The developer directly modifies, debugs, corrects, completes, replaces, or works around AI-assisted work rather than delegating the correction back to the AI.
- Update Assets: Creating or modifying a persistent resource intended to guide or improve future AI-assisted work, such as instructions, memory files, reusable prompts, project rules, documentation, tests, checklists, templates, or agent configurations. General discussion of improving a workflow or human-team rules does not count without a concrete persistent resource for AI-assisted work.

KEY DISTINCTIONS

- Plan structures how AI-assisted work will be carried out before an execution cycle; simply describing or assigning the task does not count. Teach changes the AI’s direction after execution has begun.
- Monitor requires a concrete act of observing or checking an ongoing execution; Review evaluates work, results, or behavior that has already been produced.
- Wait means the developer leaves AI work running; an agent waiting for the developer is not Wait.
- Teach delegates a correction or change back to the AI; Manual Fix means the developer makes the correction directly.
- An agent performing planning, monitoring, testing, or reviewing on its own does not automatically constitute a supervisory stage. The function must be deliberately incorporated into the developer’s supervisory process.
- Running tests or using another agent to evaluate current AI-assisted work may count as Review. Tests count as Update Assets when they are deliberately created or maintained as persistent guidance or guardrails for future AI work.

OUTPUT

Return only valid JSON:

```
{ "thread_id": "{{THREAD_ID}}",
  "has_stage": true,
  "stage_results": [
    {
      "stage": "Plan | Monitor | Wait | Review | Teach | Manual Fix | Update Assets",
```

```

"confidence": "high | mid | low",
"excerpts": [
{
"reason": "Brief reason this exact quote supports this stage",
"source_type": "post_title | post_body | comment",
"source_id": "Exact supplied source ID",
"author": "Exact supplied author",
"quote": "Exact contiguous verbatim quote"
}
]
},
"reason_if_false": ""
}

```

Include each detected stage at most once, with all supporting excerpts grouped under that stage.

If `has_stage` is true, `stage_results` must contain at least one stage and `reason_if_false` must be an empty string.

If no supervisory stage is supported, return:

```

{
"thread_id": "{{THREAD_ID}}",
"has_stage": false,
"stage_results": [],
"reason_if_false": "Brief reason no supervisory stage is supported"
}

```

Do not return markdown or any text outside the JSON object.

THREAD ID

```
{{THREAD_ID}}
```

THREAD TO ANALYZE

```
{{THREAD}}
```